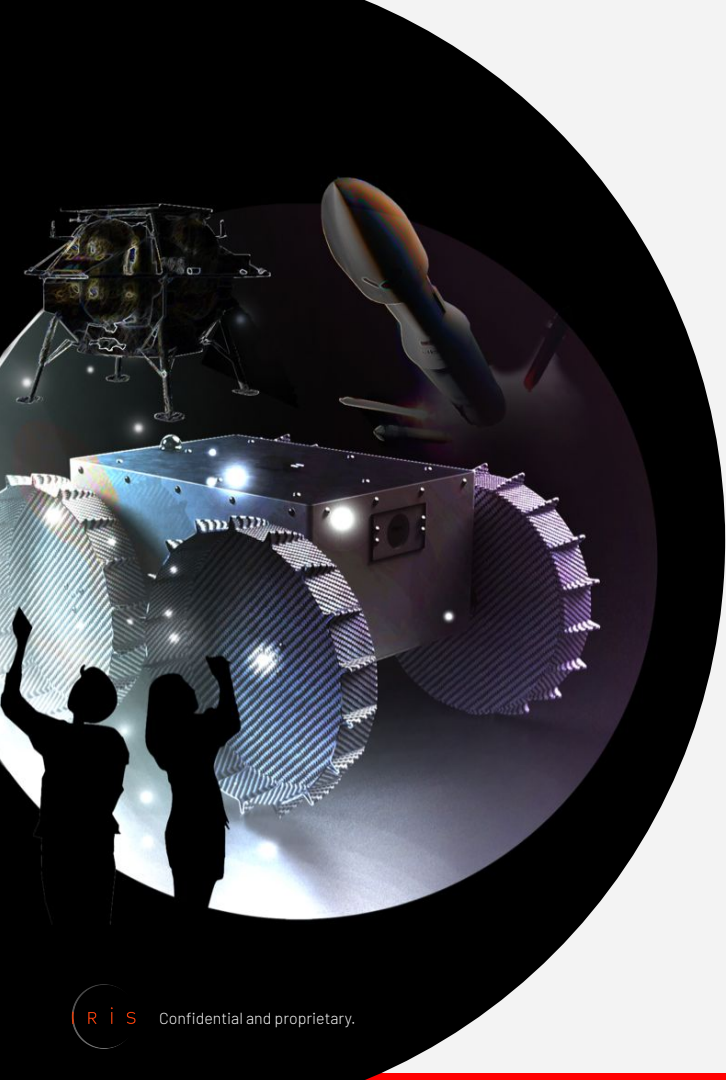




Rover Data API Intro

Using GSW GDS IPC

Connor Colombo (cwcolomb@alumni.cmu.edu)

April 22, 2023 (Updated: 10/21/2024)

03:30 PM

Agenda

- 1 Background
- 2 App Examples
- 3 Writing Code / Using the Repo
- 4 Setting up Your Machine

Background

GSW GDS IPC API

GSW

Ground Software

Collection of all mission software that runs on Earth for interacting with or understanding the Rover.

GDS

Ground Data Systems

GSW used for processing data sent to or from the Rover as well as generating new data from that data. Our core package is called **IrisBackendv3**.

IPC

Inter-Process Communication

Any system used for communicating btwn different programs. GDS contains many “apps”. Here, “IPC” refers to our (ZMQ-based ROS-like) system apps to share data.

Assumed Background

If you don't know any of these things, that's okay.

You shouldn't need to know them to understand this presentation but you will for using this.

Stick around after and I'll set you up (or DM me).

- Python
 - Pretty important. Everything uses this.
- Docker
 - Just the basics.
- Git
 - Only basics
- VSCode
 - Know how to use. Our recommended IDE. Not required.

What You **Won't** Need to Know

You may hear me mention these things, that's okay. They're abstracted. You won't need to use them.

- ZeroMQ
- ROS
- Asynchronous Python (asyncio)
- Python Environments*
 - You'll be using this but there's really nothing for you to do.
- Typed Python (**mypy**)

A Note on Python Environments

- Everything we're going to run is inside the **py_irisbackendv3** python environment.
- Basically an isolated installation of a specific python version and you're required packages.
- This is separate from your computer's **python** install (if it has one) and run separately.
- We use **pyenv**.
 - Similar to **venv** or **conda** if you've ever used those.
- **You don't really have to worry about this.**
 - Install and setup is handled by make. Just make sure this doesn't error out.
 - Just have to use **./run-script.sh** instead of **python** in the command line.
 - If you run inside a docker container, all setup including python environment is handed:
docker compose run --rm -i core-min python3 XYZ.py

Quick Rover Data Concepts

- **Downlink** (DL) Data sent from the Moon (Rover FSW) down to Earth (GSW)
- **Uplink** (UL) Data sent from Earth (GSW) up to the Moon (FSW)
- **Packet** Container of a bunch of data sent in either direction all at once.
- **Payload** A single piece of data. Most payloads come from or will end up in a Packet.
 - Also contains metadata about the payload that doesn't go to the rover.
 - Types:
 - **CommandPayload** Command telling the rover what to do.
 - **TelemetryPayload** Frequently emitted data. Happens over a "channel" (just a name thing).
 - **EventPayload** Logs / messages that happen in response to an event.
 - **FileBlockPayload** Piece of a "File" (each line of our raw images is considered a "File").
 - Some Payloads (e.g. Metachannels) are generated in GSW and stay on Earth
 - E.g. IMU sends a TelemetryPayload containing raw sensors readings, GDS converts those to angles.
- **Module** All rover data is grouped into Modules (what emits the data). Each contains Commands, Telemetry, & Events (CT&E).

Quick IPC App Concepts

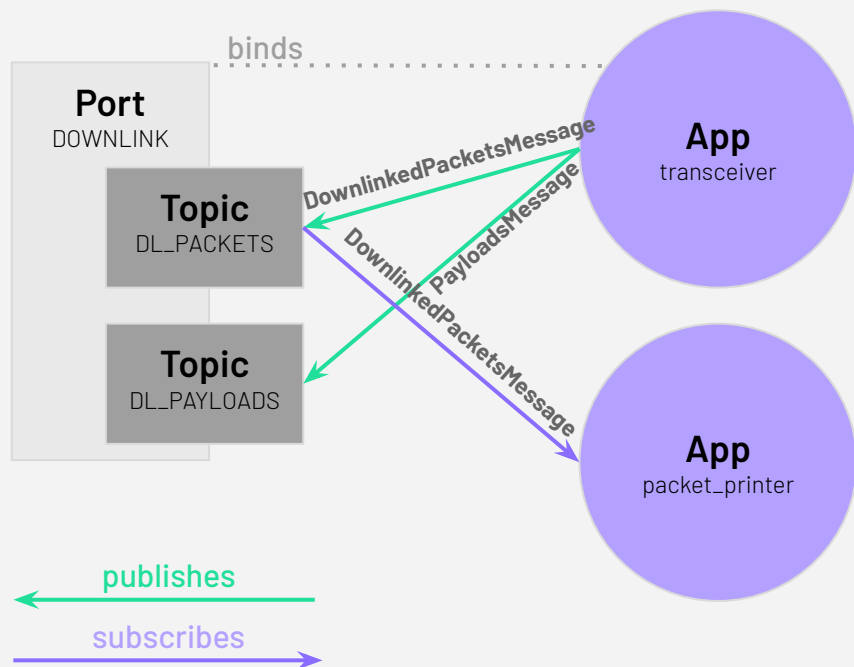
- **Port** Network communication happens through this (just an address)
- **Socket** Used to communicate through the network with a **Port**.
- **Context** Network interface, possibly containing multiple **Sockets**.

- **Message** A collection of data sent over IPC. Each message has specific content it allows.
- **Topic** Flag used for data sent on any **Port** indicating what it contains (e.g. Downlink data).
Each Topic only allows a specific message to be sent over it.
Ex: The **DL_PAYLOADS** Topic only allows **DownlinkedPayloadMessages** on it.
- **Subtopic** An optional name used to tag data within a Topic, allowing for further filtering. Rarely used.

- **Subscriber** A **Socket** which sends data to a **Topic**.
- **Publisher** A **Socket** which listens for data sent to a **Topic**.

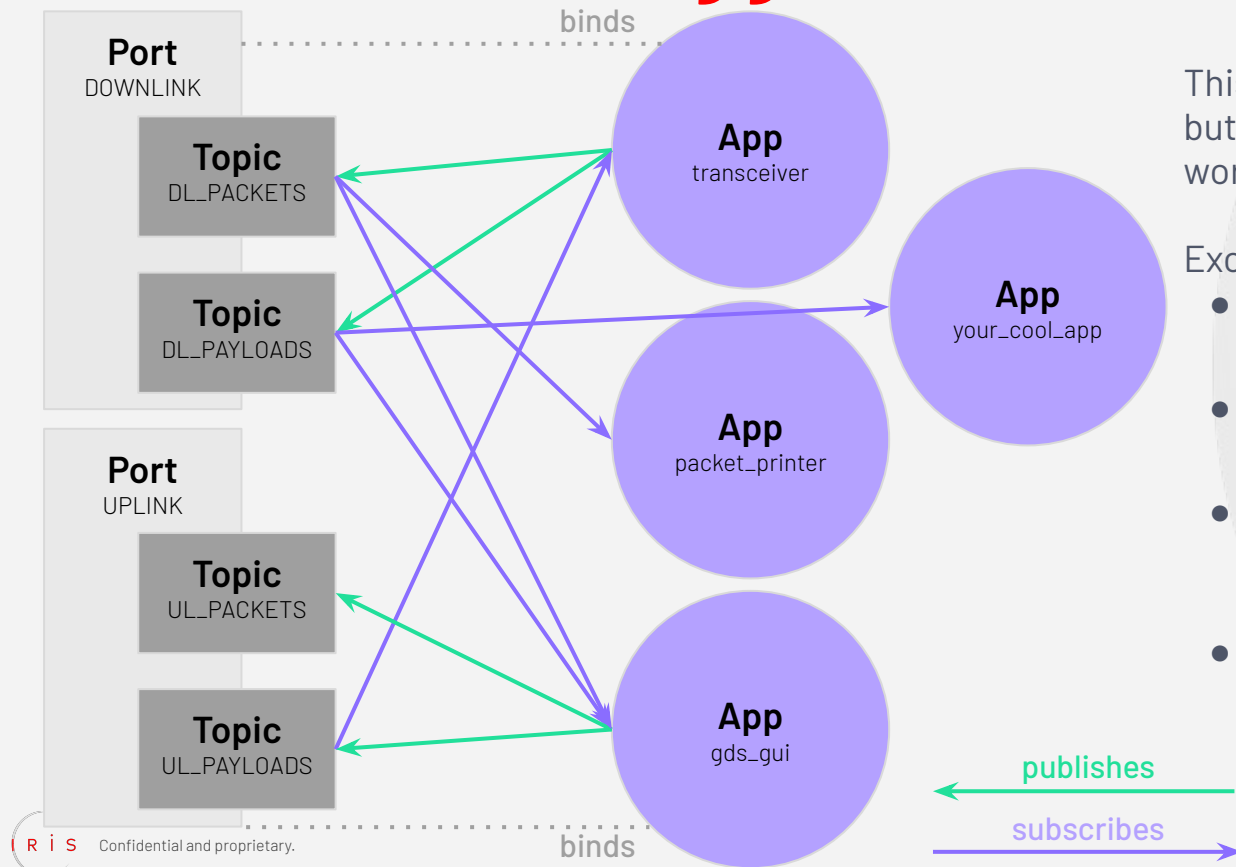
- **App** An independent program which interacts with other Apps via IPC.
- **AppManager** A wrapper that simplifies the lifecycle of an **App** and its **Sockets** and **Context**.

What is IPC (a simplified example)



- **transceiver** App publishes data on two topics.
- **packet_printer** App subscribes only the topic it cares about.
- **transceiver** binds the port
 - Each Port has exactly 1 app that **binds** the port (owns it).
 - Apps need something to bind the port to run.

What is IPC (a bigger view)

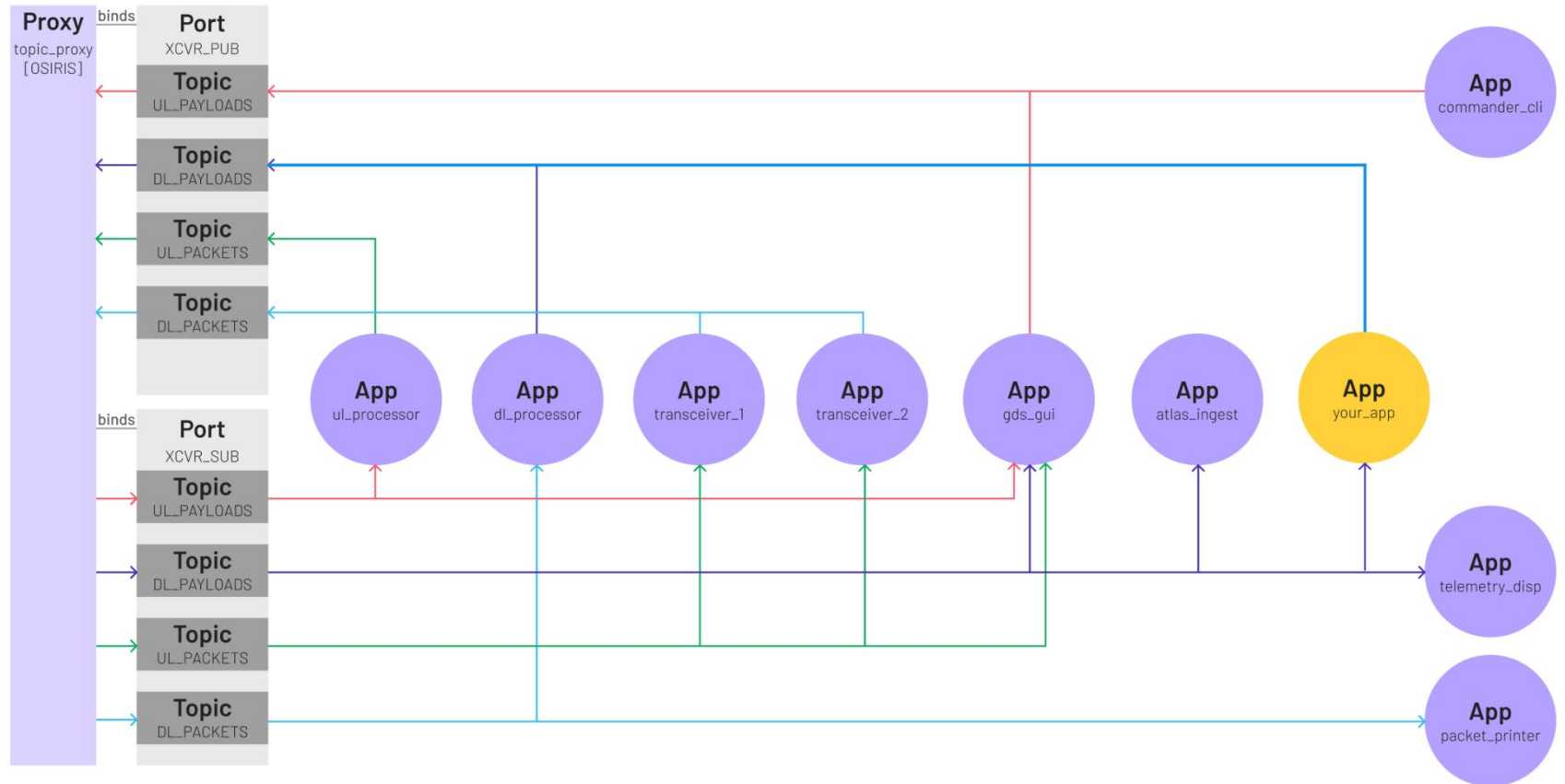


This gets complicated fast...
but it's all abstracted so you don't have to worry about it.

Except: a note on **binding**

- Each **Port** has exactly 1 app that **binds** the port (owns it).
- Apps need something to bind the port to run.
- Each port can either have:
 - 1 publisher and N subscribers
 - 1 subscriber and N publishers
- Proxy App gets around this limitation.

What is IPC (actual XCVR architecture)



What is IPC (takeaways)

- All of the above is mostly to give you context that might help you if there's an error.
- Unless you're doing something special, this is all you have to worry about:
 1. **Always** use **TRANSCIVER_SUB** Port for receiving and **TRANSCIVER_PUB** Port for publishing data.
 2. All the data you care about is probably on the **DL_PAYLOADS** Topic.
 3. **Never** bind (always use **bind=False**).
 4. When testing, **always** spin up the proxies & infrastructure (run the **make run-infra** command).
 - a. If you use Docker want to make sure all the core processes are up just **docker compose up -d core**
 5. See **./ipc_apps** directory for example apps.
 - a. **./ipc_apps/names.py** is a good example of a simple app.
 6. [Ground Software README](#) is your friend.

App Examples

Simple Consumer Example

```
Basic Rover Data App

import IrisBackendv3 as IB3
import IrisBackendv3.ipc as ipc

# Load CT&E
IB3.init_from_latest()

# Setup Your App:
payloads = ipc.BasicPayloadListener({
    IB3.codec.payload.TelemetryPayload: [
        'Imu_AccX',
        'Imu_AccY',
        'Imu_AccZ',
        'WhateverChannelsYouWant_LookAtCTE'
    ]
})

# Do Your Thing:
if __name__ == "__main__":
    for payload in payloads:
        do_something_cool_with(payload.data)
```

This is the minimal setup for an app.

- **IrisBackendv3** is our core GDS package
- **init_from_latest** loads the CT&E
- **BasicPayloadListener** is an iterable that emits when new data matching a filter arrives.
 - Only for listening (probably majority of use)
 - Available in this week's release

Socket-Based Consumer App Example

scripts/testing/ipc_demo/ipc_app_sync_sub.py (look here for the latest version)

Using sockets and **AppManager** allows you to migrate to more advanced setups later.

```
IPC Subscriber App

import IrisBackendv3 as IB3
import IrisBackendv3.ipc as ipc
IB3.init_from_latest()

# Setup:
manager = ipc.IpcAppManagerSync(socket_specs={
    'sub': ipc.SocketSpec(
        sock_type=ipc.SocketType.SUBSCRIBER,
        port=ipc.Port.TRANSCIEVER,
        topics=[ipc.Topic.DL_PACKETS, ipc.Topic.UL_PACKET],
        bind=True
    )
})

# Run:
for _ in range(10):
    raw, msg = manager.read_msg('sub')
    print(
        f"{raw.topic=}, {raw.subtopic_bytes=} with {msg.content!s}"
    )
```


Async Consumer App Example

[scripts/testing/ipc_demo/ipc_app_async_sub.py](#) (look here for the latest version)

```
1 > """...
11 import asyncio
12 from typing import ClassVar
13
14 import IrisBackendv3 as IB3
15 import IrisBackendv3.ipc as ipc
16 from IrisBackendv3.ipc.messages import DownlinkedPacketsMessage
17
18 IB3.init_from_latest()
```

```
20 class Sub(ipc.SocketTopicHandlerAsync['Sub']):
21     """A demo Subscriber that only cares about 1 topic and ends everything
22     after it gets 10 messages."""
23     _raise_on_unhandled_topics: ClassVar[bool] = False
24     _require_unhandled_topic_handler: ClassVar[bool] = False
25     # Decorator shorthand (also plays more nicely with syntax highlighting):
26     topic_handler = ipc.SocketTopicHandlerAsync['Sub'].TopicHandlerFactory()
27
28     rx_count: int # simple example of maintaining an internal state
29
30     def __init__(self, *args, **kwargs) -> None:
31         self.rx_count = 0
32         super().__init__(*args, **kwargs)
33
34     @topic_handler
35     async def downlink_handler(
36         self,
37         manager: ipc.IpcAppManagerAsync,
38         payload: ipc.IpcPayload
39     ) -> None:
40         # Decode the message:
41         msg: ipc.InterProcessMessage = payload.message
42         # Make sure we got the message we expected (not just any message):
43         msg = ipc.guard_msg(msg, DownlinkedPacketsMessage)
44         print(
45             f"{payload.topic=}, {payload.subtopic_bytes=} "
46             f"with {msg.content.simple_str()}"
47         )
48         self.rx_count += 1
49         if self.rx_count == 10:
50             # We're done. Yeet out of here:
51             raise ipc.IpcEndAppRequest("Got 10. Thanks!")
52
53     _topic_handlers: ClassVar[ipc.SocketTopicHandlerAsync.TopicHandlerRegistry] = {}
54     ipc.Topic.DL_PACKETS: downlink_handler
55
56
```

```
manager = IB3.ipc.IpcAppManagerAsync(socket_specs={
    'sub': IB3.ipc.SocketSpec(
        sock_type=IB3.ipc.SocketType.SUBSCRIBER,
        port=IB3.ipc.Port.TRANSCIVER,
        topics=Sub.TOPICS(),
        rx_handler=Sub(),
        bind=True
    )
})

asyncio.run(manager.run())
```

Simple Producer Example (Live Demo)

[scripts/testing/ipc_demo/ipc_sync_pub.py](#) (look here for the latest version)

```
12 import time
13 import IrisBackendv3 as IB3
14 import IrisBackendv3.ipc as ipc
15 from IrisBackendv3.ipc.messages import (
16     DownlinkedPacketsMessage, UplinkPacketRequestMessage,
17     DownlinkedPacketsContent, UplinkPacketRequestContent
18 )
19 from IrisBackendv3.transceiver.logging import logger_setConsoleLevel as xcvrLoggerLevel
20
21 IB3.init_from_latest()
22
23 # Load data:
24 xcvrLoggerLevel('NOTICE')
25 > xcvr = IB3.transceiver.prebuilts.build_xcwr_by_name( ...
26 xcvr.begin()
27 time.sleep(1) # wait for XCVR to fill up with some packets
28 packets = xcvr.read()
29
30 # Create socket:
31 context = IB3.ipc.create_context()
32 socket = IB3.ipc.create_socket(
33     context,
34     IB3.ipc.SocketType.PUBLISHER,
35     IB3.ipc.Port.TRANSCIVER,
36     bind=False
37 )
38
39 # Wait for sub to connect:
40 print("Waiting 5 sec . . .")
41 time.sleep(5) # much longer than sub will take... but that's the point
42
```

```
49 # Send 10 packets:
50 print("Sending 10 messages to two topics:")
51 for i in range(10):
52     # Send 1 msg to the topic being listened to:
53     msg = DownlinkedPacketsMessage(DownlinkedPacketsContent(
54         packets=packets[:i] # Send as many packets as there are
55     ))
56     ipc.send_to(socket, msg, subtopic=b'packets', topic=ipc.Topic.DL_PACKETS)
57     print(f"Sent to {ipc.Topic.DL_PACKETS}: {i} packets.")
58
59     # Send another to a different topic (to make sure it doesn't go through):
60     msg2 = UplinkPacketRequestMessage(UplinkPacketRequestContent(
61         packet=packets[0]
62     ))
63     ipc.send_to(socket, msg2, subtopic=b'packets', topic=ipc.Topic.UL_PACKET)
64     print(f"Send to {ipc.Topic.UL_PACKET}: 1 packet.")
65     # Wait before sending the next one:
66     time.sleep(1)
67
68 # Teardown:
69 print("Tearing down . . .")
70 socket.close()
71 context.destroy()
72
```

Real Consumer App Example (Live Demo)

`ipc_apps/packet_printer.py` (look here for the latest version)

```
15 import asyncio
16 import argparse
17 from typing import Callable, ClassVar
18
19 > if __name__ == "__main__":
20
21     import IrisBackendV3 as IB3
22     import IrisBackendV3.ipc as ipc
23     from IrisBackendV3.ipc.messages import (
24         DownlinkedPacketsMessage,
25         UplinkPacketRequestMessage
26     )
27
28     from IrisBackendV3.utils import console_display
29
30 > parser = argparse.ArgumentParser(description=(
31     """
32     You, last month via PR #213 * IPC: Packet Printer
33     """))
34
35 > def get_opts():
36     You, last month via PR #213 * IPC: Packet Printer
37     opts = get_opts()
38
39 # Set app name config:
40 app = ipc.IpcAppHelper(opts.name)
41 app.setLogLevel(opts.log_level)
42 IB3.init_from_latest()
43
44 > def packet_to_log_level(packet: IB3.codec.packet.
131
132 class Sub(ipc.SocketTopicHandlerAsync['Sub']):
133     You, last month via PR #213 * IPC: Packet Printer
134     """A demo Subscriber that only cares about 1 topic and ends everything--
135     _raise_on_unhandled_topics: ClassVar[bool] = False
136     _require_unhandled_topic_handler: ClassVar[bool] = False
137     # Decorator shorthand (also plays more nicely with syntax highlighting):
138     topic_handler = ipc.SocketTopicHandlerAsync['Sub'].TopicHandlerFactory()
139
140     def __init__(self, *args, **kwargs) -> None:
141
142     @topic_handler
143     async def downlink_handler(
144         self,
145         manager: ipc.IpcAppManagerAsync,
146         payload: ipc.IpcPayload
147     ) -> None:
148         # Decode the message:
149         msg = ipc.guard_msg(payload.message, DownlinkedPacketsMessage)
150
151         # Print each packet:
152         for packet in msg.content.packets:
153
154     @topic_handler
155     async def uplink_handler(
156         self,
157         manager: ipc.IpcAppManagerAsync,
158         payload: ipc.IpcPayload
159     ) -> None:
160         # Decode the message:
161         msg = ipc.guard_msg(payload.message, UplinkPacketRequestMessage)
162         packet = msg.content.packet
163
164         # Print the packet and the request metadata:
165         app.logger.notice(
166
167     _topic_handlers: ClassVar[ipc.SocketTopicHandlerAsync.TopicHandlerRegistry] = {
168         ipc.Topic.DL_PACKETS: downlink_handler,
169         ipc.Topic.UL_PACKET: uplink_handler
170     }
171
172 manager = ipc.IpcAppManagerAsync(socket_specs={
173     'sub': ipc.SocketSpec(
174         sock_type=ipc.SocketType.SUBSCRIBER,
175         port=ipc.Port.TRANSCIVER,
176         topics=Sub.TOPICS(),
177         rx_handler=Sub(),
178         bind=opts.bind
179     )
180 })
181
182 > if __name__ == "__main__":
183     asyncio.run(manager.run())
184
```

Producer App Example (Live Demo)

`scripts/testing/ipc_demo/pcap_xcvt.py` *(look here for the latest version)*

Where to Find Available Data (Command Telemetry & Events)

RC8 (CT&E at bottom): <https://github.com/PlanetaryRobotics/IrisRoverPackage/releases/tag/rc8>

Direct link: [RC8 CT&E](#). Check [Releases](#) for more recent releases or see [README](#) for how to build new CT&E.

Index of all Modules:

Name (major ones highlighted)	General Description
BlockDriver	Drives FPrime cycles.
RateGroupDriver	Triggers RateGroups.
ActiveRateGroup	
RateGroupLowFreq	Low-frequency ticker.
ActiveRateGroup	
RateGroupMedFreq	Medium-frequency ticker.
ActiveRateGroup	
RateGroupHIFreq	High-frequency ticker.
CubeRoverTime	Manages time inside Hercules.
TimChan	Routes telemetry between components.
CommandDispatcher	Handles making sure Hercules commands go to the right place.
GroundInterface	Metadata about data sent to and from Hercules (from Ground). Also where names and messages are emitted.
NetworkManager	Manages and maintains communications between Hercules and the Radio processor. Mostly signal strength data.
ActiveLogger	Routes logs/events between components. Allows for filtering certain logs out.
Navigation	Highlevel planning and administration of movement operations.
MotorControl	Manages communications between Hercules and the Motor Controller MCUs. Monitors motor, motor driver, and motor controller state.
Imu	Communicates with the 6-DOF IMU (acceleration & gyro).
WatchDogInterface	Manages communications between Hercules and the Watchdog. Also where on-surface thermal measurements come from.
Camera	Interfaces with Camera via FPGA and Camera flash. Takes pictures and manages image storage.
Peregrine	Data from the lander (mostly deck temperatures and comms status).
RadioGround	Health and signal data straight from the Radio processor.
WatchdogHeartbeatTvac	Handler for special packets containing basic aliveness and thermal information from the Watchdog.
WatchdogHeartbeat	Handler for special packets containing basic aliveness information from the Watchdog.
WatchdogDetailedStatus	Handler for special packets containing information on the full system state administered by the Watchdog. Power, battery, deployment, critical thermals.
WatchdogCommandResponse	Handler for special packets containing command responses from the Watchdog.

Radio_RadioGround (RC8) x 410M, 4 Command, 7 Telemetry Channel, 11 Events

Command
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command Args Table:				
Arg	Name	Type	Bytes (max)	Comment
0	last_to_echo	VARSTRING_255	257	None

Command: RadioGround_RouteRadioConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command Args Table:				
Arg	Name	Type	Bytes (max)	Comment
0	confirm_by_typing_RESET	STRING5	7	None

Command: RadioGround_ExtStatusConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command: RadioGround_EnterStasis(confirm_by_typing_STASIS)	
Opcode:	0xEF5 (6145)
Mnemonic:	EnterStasis

Command: RadioGround_ExtStatusConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command: RadioGround_ExitStatus(confirm_by typing_STASIS)				
Opcode:	0xEF0E (61270)			
Mnemonic:	ExitStack			
Command Args Table:				
Arg	Name	Type	Bytes (min)	Comment

Command: RadioGround_WatchdogToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Opcode:	0x6FD0 (61433)
Mnemonic:	WriteJsr

Command Args Table:				
Arg	Name	Type	Bytes (max)	Comment
0	text to echo	VARSTRING_255	255	None

Command: RadioGround_HeartbeatToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command Args Table:				
Arg	Name	Type	Bytes (max)	Comment
0	text to echo	VARSTRING_255	257	None

Command: RadioGround_DetailedStatusToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command	Arg	Name	Type	Bytes (min)	Comment
0	0	last_in_ack	VAIRFREQ_255	20	None
	1	last_in_ack	VAIRFREQ_255	20	None
	2	last_in_ack	VAIRFREQ_255	20	None

Command
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

0x0F01 (01198)	StateAddr	ENUM	4	None
0x0F02 (01199)	UptimeTicks	U32	4	None
0x0F03 (01200)	UptimeMs	U32	4	None
0x0F04 (01201)	UpgrPcketsCount	U32	4	None
0x0F05 (01202)	BacklogPacketsCount	U32	4	None
0x0F06 (01203)	HerculesUpgrPacketsCount	U8	1	None

Command: RadioGround_RouteRadioConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

INIT	1	0x0001	INITIALIZED. Trying to turn radio antenna on.
WIFON	2	0x0002	WIFI_ON. Radio antenna is on. Trying to connect to the network.
CONN	3	0x0003	CONNECTED. Connected to WiFi network. Attempting to establish

Command: RadioGround_ExtStatusConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command	Arg	Name	Type	Bytes (min)	Comment
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS	STRINGS		None
confirm_by_spine	0	confirm_by_spine_STATUS	STRINGS		None
	1	confirm_by_spine_STATUS			

Command: RadioGround_ExtStatusConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command: RadioGround_WatchdogToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command: RadioGround_HeartbeatToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command Args Table				
Arg	Name	Type	Bytes (min)	Comment
0	last_in_ack	VAIRFREQ_255	20	None

Command: RadioGround_DetailedStatusToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Command	Arg	Name	Type	Bytes (min)	Comment
0	last_in_ack	VAIRFREQ_255	20	None	

Command
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Format String:	Radio Heartbeat: %s with %s RSSI at %s.			
Event Args Table:				
Arg	Name	Type	Bytes (max)	Comment
0	state_addr	ENUM	4	None
1	ssi	16	2	None

Command: RadioGround_RouteRadioConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

		BOOT	0	0x0000	BOOT. Radio just booted.
		INIT	1	0x0001	INITIALIZED. Trying to turn on antenna on.
Fi		WIFON	2	0x0002	WIFI_ON. Radio antenna is on. Trying to connect to the network.

Command: RadioGround_ExtStatusConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

	CRPT	255	0x00FF	The abbreviation received didn't match any known Radio State Abbrev. See raw data for actual bytes received.

Command: RadioGround_ExtStatusConfirmBySpine (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Event Args Table				
Arg	Name	Type	Bytes (max)	Comment
0	message	STRING38	41	None
[Event: Echo[echoed_text]]				
Decoded: C:\E02\J01590				

Command: RadioGround_WatchdogToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

Arg	Name	Type	Bytes (max)	Comment
0	echoed_text	VARIABLE_STRING_255	255	None

Event: GotCommand[command, string_of_command_data]

Opcode: 0x0F to 0x100

Command: RadioGround_HeartbeatToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

0	command	ENUM	4	None
1	string of command data	VARSTRING 255	257	None

Event Arg Enum Table: command (8)				
Name	Value	Hex	Comment	
FC40	0	0x0000	Fetch back the stream index	

Command: RadioGround_DetailedStatusToWatch (RC8)
Depends: RadioGround_ExtStatusToWatch
Depends: RadioGround_ExtStatusToWatch

	HERCULES_DM	221	0x0000	Indicate the given user is Hercules Radio Direct Message.
	CORRUPTED	255	0x00FF	Given ID doesn't match any known IDs. Likely corrupted.

Using GDS to Examine DataStandards (CT&E)

The standards are available in python, so you can use them instead of CT&E excel if you want:

```
(py_irisbackendv3) (base) conncolomboc@MacBook-Pro-4 GroundSoftware % ./run-script.sh
Python 3.10.8 (main, Mar 2 2023, 15:21:58) [Clang 13.1.6 (clang-1316.0.21.2.5)] on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>>
>>> import IrisBackendv3 as IB3
WARNING: No IPv4 address found on anpi0 !
WARNING: No IPv4 address found on anpi1 !
WARNING: more No IPv4 address found on anpi2 !
>>> IB3.init_from_latest()
2023-04-22 15:20:55: INFO      Successfully Loaded 22 Modules with 80 Commands, 188 Channels, and 72 Eve
>>> standards = IB3.get_codec_standards()
>>> standards.modules.names
['BlockDriver', 'RateGroupDriver', 'ActiveRateGroup-RateGroupLowFreq', 'ActiveRateGroup-RateGroupMedFre
er', 'ActiveLogger', 'Navigation', 'MotorControl', 'Imu', 'WatchDogInterface', 'Camera', 'Peregrine', '
>>> imu = standards.modules['Imu']
>>> imu.telemetry.names
['XAcc', 'YAcc', 'ZAcc', 'XAng', 'YAng', 'ZAng']
>>> █
```

This allows you to perform some more advanced searches (e.g. every telemetry channel containing 'temp'):

```
>>> print('\n'.join(f'{m.name}: {t.name}' for m in standards.modules.vals for t in m.telemetry.vals if 'temp' in t.name.lower()))
Peregrine: DeckD1TempKelvin
Peregrine: DeckD2TempKelvin
Peregrine: DeckD3TempKelvin
WatchdogHeartbeatTvac: AdcTempRaw
WatchdogHeartbeatTvac: AdcTempKelvin
WatchdogHeartbeatTvac: FuelTempRaw
WatchdogHeartbeatTvac: FuelTempKelvin
WatchdogHeartbeat: BattAdcTempRaw
WatchdogHeartbeat: BattAdcTempKelvin
```

Running Scripts

- **Always run from the IrisRoverPackage/Apps/GroundSoftware directory.**
- Run python script like normal except use **./run-script.sh XYZ.py**

```
./run-script.sh ./scripts/tiny_apps/datastandards_build_cache.py -l NOTICE
WARNING: No IPv4 address found on anpi0 !
WARNING: No IPv4 address found on anpi1 !
WARNING: more No IPv4 address found on anpi2 !
2023-04-22 14:56:29: NOTICE    Building DataStandards . . .
2023-04-22 14:56:29: NOTICE    Multiple <instance>s of <component> ActiveRateGroup found
created for each instance. Instances were found at lines [30, 31, 32] in Topology file
2023-04-22 14:56:29: NOTICE    Adding 6 Prebuilt Modules . . .
2023-04-22 14:56:29: NOTICE    Caching DataStandards . . .
2023-04-22 14:56:29: SUCCESS    DataStandards Cached to `Users/connorcolombo/Documents/
```

- You can also run inside a docker container with
docker compose run --rm -i core-min python3 XYZ.py

Testing without a Rover with Docker

- Run your script
- Run **docker compose up -d pcap-data** or **docker compose up -d fm1-data** (if you have FM1 data)
- [Ground Software README](#) is your friend for more details.

Testing without a Rover (Manual Test Data)

- Run your script in one terminal
- Run the **packet capture transceiver** (PCAP XCVR) in another window.
./run-script.sh scripts/testing/ipc_demo/pcap_xcvr.py -x PCAP-RC9.5.3-2MIN-IMU -t 588 -loop
 - **-x** Which pre-built transceiver to run (which data capture to use)
 - You can pre-configure your own if you have special test data.
 - This is two minutes of recent telem (204 packets) from RC9.5.3 with IMU movement.
 - **-t** Number of seconds between each packet being emitted.
 - This is a simple script that emits data at fixed interval.
 - 500-5000 is fairly flight-like.
 - **-loop** (or **-no-loop**) Whether or not to loop the PCAP for infinite playback.
- May take a minute to extract all the data and start replay.
- You can just leave this running in the background.
- **Ctrl+C** to quit.

Testing without a Rover (Test Data)

Finding More Examples

- Simple use-case demos / tests can be found in **scripts/testing/ipc_demo**
- Implemented apps can be found in **ipc_apps**
- Most examples contain notes at the top of the script on how to run them / what to run alongside them.

Writing Code / Using the Repository

Repository Structure

Name
..
.vscode
IrisBackendv3
config
ipc_apps
out
scripts
target
test-data
test

- Useful workspace settings for VSCode
- The core GDS package. You'll use this. **Don't** edit it.
- Configs for our Rover (work in progress).
- Some core IPC Apps used in "production" (FSW test pipeline).
- Things GDS produces. Not tracked in **git** except for milestones.
- Legacy, testing, and not-ready-for-production python scripts.
- Infrastructure
- Data used to test the GDS. Largely Rover-Data logs.
- End-to-End test setup. WIP.

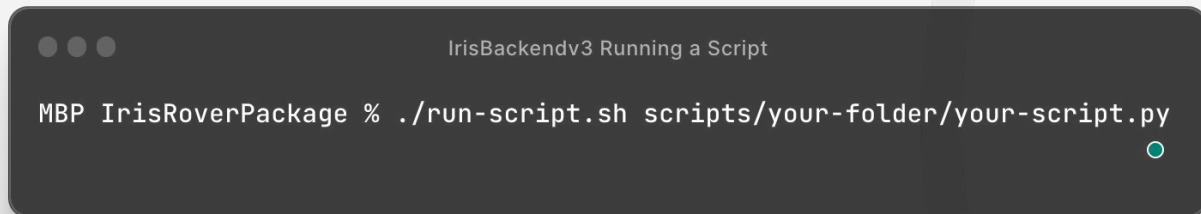
IrisBackendv3 Structure

Things you'll encounter inside the **IrisBackendv3** package.

- **codec** (CODEC) Encodes and decodes packets, payloads, and data from raw binary
- **config** (CFG) Tools for managing rover configuration data.
- **data_standards** (DS) Tools for loading and saving CT&E. Not the CT&E itself.
- **ipc** (IPC) Standardized interface for simple bring-up of IPC apps using data from the **codec**.
- **logs** (LOG) Common logging interface.
- **meta** (META) Extension to DataStandards for definitions of "meta" fields generated by the GDS
- **storage** (STOR) Manages archiving data locally (and in the cloud eventually).
- **transceiver** (XCVR) Handles sending data between the Rover and the **codec** over a variety of interfaces.
- **utils** (UTIL) Common utilities. Mostly special data-structures and formatting tools.

Adding & Running Code

- Put your code in a folder in **scripts** while you're working on it.
- Once it's ready for production, we'll move it to **ipc_apps**.
- Run using **./run-script.sh**

A terminal window with a dark background and light text. The title bar at the top reads "IrisBackendv3 Running a Script". The terminal content shows a prompt "MBP IrisRoverPackage %" followed by the command `./run-script.sh scripts/your-folder/your-script.py`. A green cursor is positioned at the end of the command line.

```
IrisBackendv3 Running a Script  
MBP IrisRoverPackage % ./run-script.sh scripts/your-folder/your-script.py
```

- This is just a wrapper for **pyenv exec python3 ...** but is shorter and allows you to use tab-complete.
- Use **./run-script.sh** in exactly the same way you would use **python/python3**
 - Supports command line arguments, calling by itself, etc.

Adding Requirements

- If you need to add a python requirement, **don't just install it via command line.**
- Add it to **requirements.txt**
 - Consider adding a version specifier

```
1  mypy
2  typing_extensions>=4.0.0
3  typeguard~=2.13.3
4  ipykernel
5  scapy
6  python-pcapng
7  numba~=0.56.4
8  numpy<1.24
9  matplotlib
10 seaborn~=0.12.2
11 pandas
12 tabulate
13 styleframe
14 xlswriter
```

- Run **make update** which makes sure everything is up to date. (Also **docker compose build** again)

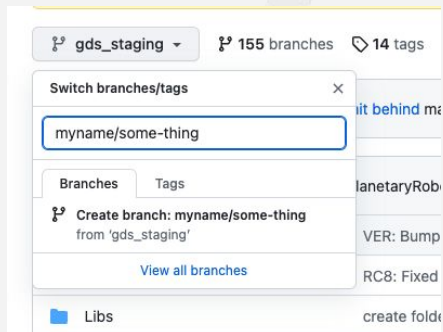
```
(py_irisbackendv3) (base) connorcolombo@MacBook-Pro-4 GroundSoftware % make update
pyenv exec python -m pip install -r requirements.txt;
Requirement already satisfied: mypy in /Users/connorcolombo/.pyenv/versions/3.10.8/envs/py_iri
```



How to Get Set Up

How to Get It

- Clone from <https://github.com/PlanetaryRobotics/IrisRoverPackage>
- If developing, checkout **gds_staging** branch
 - Basically just **release_candidate** but safe to push here
 - **Don't** just push here
 - **Do** create your own feature branch
- Go to **Apps/GroundSoftware**



How to Set Up

- These instructions (and more details) can be found in [Apps/GroundSoftware/README.md](#)
- Either use **Docker** and just **docker compose build** OR set up locally:
- [Minimal Setup Commands](#) (**make init** takes a while to run and requires your input occasionally):

```
IrisBackendv3 Setup

MBP IrisRoverPackage % cd Apps/GroundSoftware
MBP GroundSoftware % ./__wsl-ubuntu-setup.sh
MBP GroundSoftware % make init
MBP GroundSoftware % ./__dev-setup.sh
```

```
(py_irisbackendv3) (base) conncorcolombo@MacBook-Pro-4 GroundSoftware % make standards
./run-script.sh ./scripts/tiny_apps/datastandards_build_cache.py -l NOTICE
WARNING: No IPv4 address found on anpi0 !
WARNING: No IPv4 address found on anpi1 !
WARNING: more No IPv4 address found on anpi2 !
2023-04-22 14:15:41: NOTICE Building DataStandards . . .
2023-04-22 14:15:41: NOTICE Multiple <instance>s of <component> ActiveRateGroup found in FPrime Topology for module defined at 9. Since all instance names and base_ids are unique, new standards objects will be
created for each instance. Instances were found at lines [30, 31, 32] in Topology file ../FlightSoftware/fprime/./CubeRover/Top/CubeRoverTopologyAppAi.xml
2023-04-22 14:15:42: NOTICE Adding 6 Prebuilt Modules . . .
2023-04-22 14:15:42: NOTICE Caching DataStandards . . .
2023-04-22 14:15:42: SUCCESS DataStandards Cached to ~/Users/conncorcolombo/Documents/PKBACK/GitHub/CubeRoverPackage/Apps/GroundSoftware/out/DataStandardsCache/IBv3_datastandards_01GYN2VM7NEEM64KAS0H0VMYFQ.dsc`
(py_irisbackendv3) (base) conncorcolombo@MacBook-Pro-4 GroundSoftware %
```

Notes about OS Environment

- On Linux (where we use this most)
 - Follow setup
 - Run `./__wsl-ubuntu-setup.sh`
- On Mac (where this was developed)
 - Follow normal instructions.
 - **Don't** run `./__wsl-ubuntu-setup.sh`
 - You'll likely miss some OS-level requirements (likely for **matplotlib**).
 - All are pretty basic to Google.
 - Fix and re-run **make init** (or just **make update** if the issue was with a python requirement).
 - DM me if you can't get past something
- On Windows
 - Use **Ubuntu 18.04 on WSL2**
 - Follow Set-Up Notes Here.
 - Pay attention to details like when to run as admin.
 - We use this a lot but set up has been tricky on some machines.
 - DM me for help. Pretty much everyone has had some sort of issue with WSL.
 - Run `./__wsl-ubuntu-setup.sh`

Running Unit Tests

- If you're worried something's broken / not set-up, you can re-run unit tests with **make run-tests**
 - Not comprehensive yet, but covers 63% of code-base and is sufficient to determine if your setup & IPC works.
 - Also tests IPC networking.

```
(py_irisbackendv3) (base) connorcolombo@MacBook-Pro-4 GroundSoftware % make run-tests
./run-script.sh -m pytest --cov=IrisBackendv3

===== test session starts =====
platform darwin -- Python 3.10.8, pytest-7.2.1, pluggy-1.0.0
rootdir: /Users/connorcolombo/Documents/PKBACK/GitHub/CubeRoverPackage/Apps/GroundSoftware, configfile: pytest.ini, testpaths: IrisBackendv3
plugins: asyncio-0.20.3, dash-2.8.1, typeguard-2.13.3, cov-4.0.0
asyncio: mode=strict
collected 107 items

IrisBackendv3/data_standards/_test_data_standards.py ..... [ 5%]
IrisBackendv3/data_standards/_test_module.py ..... [ 24%]
IrisBackendv3/ipc/_test_ipc_encode_decode_legacy.py .. [ 26%]
IrisBackendv3/utls/_test_crc.py . [ 27%]
IrisBackendv3/utls/_test_nameiddict.py ..... [ 94%]
IrisBackendv3/utls/basic.py ..... [ 99%]
IrisBackendv3/utls/crc.py . [100%]

TOTAL 7551 2781 63%

===== 107 passed in 38.07s =====
(py_irisbackendv3) (base) connorcolombo@MacBook-Pro-4 GroundSoftware %
```

Any Questions?

Join #gds—consumers on Slack